

DSA STUDY BLUEPRINT SERIES

77. Gas Station Greedy Approach Leetcode 134

Greedy Gas Station Circuit Completion Loops

Section 08 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Travel circular gas station loop without empty tanks.
- **Greedy Strategy:** If total gas $\geq$ total cost, a solution always exists.
- Reset start index when running balance drops below zero.

Memory & Execution Blueprint

Net Gas: [-1, -1, 2] $\Rightarrow$ Starts at index 2 where balance becomes positive.

C++ Implementation Reference

Standard code layout and syntax

```
int canCompleteCircuit(vector& gas, vector& cost) {  
    int totalGas = 0, totalCost = 0, tank = 0, start = 0;  
    for (int i = 0; i < gas.size(); i++) {  
        totalGas += gas[i]; totalCost += cost[i];  
        tank += gas[i] - cost[i];  
        if (tank < 0) { start = i + 1; tank = 0; }  
    }  
    return (totalGas >= totalCost) ? start : -1;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(1)$

Key Practices & Gotchas

- **Important:** Requires only a single loop pass without cyclic search step checking.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace circular queue pointer wrapping when capacity = 3:

Operation	Queue data layout	Pointers [front, rear]	Size count
enqueue(10)	[10, _, _]	[0, 0]	1
enqueue(20)	[10, 20, _]	[0, 1]	2
dequeue()	[_ , 20, _]	[1, 1]	1 (wraps correctly)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Wrap-around indices:** Modulo indexing updates circular buffer limits dynamically.
- **Deque mechanics:** Doubly ended queues allow fast $O(1)$ front and rear pushes and pops.
- **Related LeetCode Problems:** #239 Sliding Window Maximum, #622 Design Circular Queue, #225 Implement Stack using Queues.